Как работает пайплайн

От разбора APK до поиска устойчивого K: распознавание русской речи, диаризация и локальная LLM на устройстве

Android-приложение **com.example.baseapp** · отчёт от 8 ноября 2026

Коротко. Приложение полностью офлайн: GigaAM v3 распознаёт русскую речь, pyannote + ERes2Net делят её на спикеров, каждому слову присваивается номер говорящего, реплики склеиваются в диалог, локальная Qwen2.5 1.5B расставляет пунктуацию. Наружу отдаётся JSON `{transcript:[...]}` с целочисленным `speaker_id`.

Проверено на устройстве: iQOO 15R (Android 16, arm64), диалог 56 с — 2 спикера, 119 слов, 4 реплики, корректная пунктуация от LLM.

Содержание

1. Общая схема пайплайна
2. Что внутри APK
3. Декодирование аудио
4. Нарезка на сегменты
5. Распознавание речи: GigaAM v3
6. Диаризация и автоматическое число спикеров
7. Назначение спикера каждому слову
8. Склейка реплик и JSON
9. Локальная LLM и системный промпт
10. Метрики и результаты
11. Ограничения
12. Где что лежит в коде

1. Общая схема

Пайплайн линейный, но две ветки — распознавание и диаризация — идут независимо по одному и тому же буферу 16 кГц и встречаются на шаге назначения спикеров словам. Это важно: ASR не нужно ждать диаризацию, а диаризация не зависит от текста.

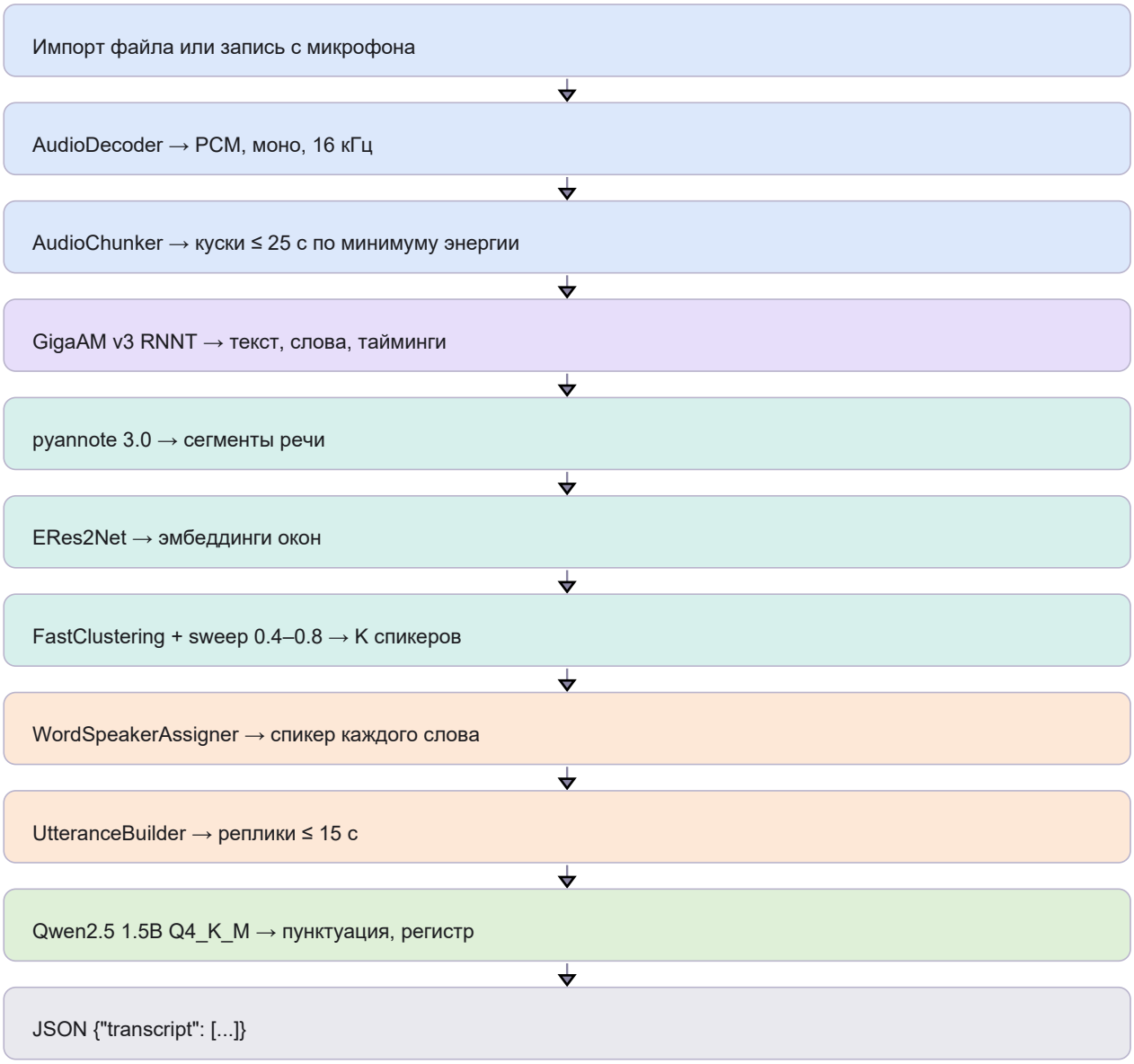


Рис. 1. Путь одного файла: от импорта до JSON.

Прогресс в интерфейсе привязан к этим шагам: 2 % — старт, 10 % — аудио декодировано, 70 % — ASR готов, 92 % — диаризация готова, 100 % — LLM закончила правку. Поэтому полоса прогресса «замирает» на 92 % дольше всего: именно там работает 1.5-миллиардная модель.

2. Что внутри APK

Приложение собрано под `minSdk 24` и содержит две ABI: `arm64-v8a` и `x86_64`. Готовый debug-APK занимает 93 863 438 байт. Внутри — 14 dex-файлов и 16 нативных библиотек; моделей в APK нет.

Ключевые нативные библиотеки

Библиотека	Размер	Зачем

libonnxruntime.so	21,2 МБ	инференс ONNX-моделей
libsherpa-onnx-jni.so	4,4 МБ	JNI-мост Kotlin → C++ sherpa-onnx
libsherpa-onnx-c-api.so	4,1 МБ	C-API sherpa-onnx
libllama.so + libggml*.so	4,1 МБ	llama.cpp для GGUF
libllama_jni.so	0,04 МБ	JNI-мост к llama.cpp
libc++_shared.so	1,2 МБ	рантайм C++

Цепочка вызовов для ASR и диаризации: Kotlin-обёртка sherpa-onnx → JNI → C++ sherpa-onnx → ONNX Runtime → int8-граф. Для LLM: Kotlin-адаптер `LlamaCppLocalLlm` → AAR `llama-android` → llama.cpp на CPU/NEON. Ни облака, ни сетевых вызовов в момент распознавания нет.

Модели: скачиваются один раз

Что	Путь в filesDir	Размер
GigaAM v3 (RNNT)	models/gigaam-v3/	229 МБ
Сегментатор pyannote 3.0	models/speaker-diarization/model.onnx	5,7 МБ
Эмбеддинги ERes2Net	models/speaker-diarization/…eres2net….onnx	37,8 МБ
Qwen2.5 1.5B Q4_K_M	models/llm/qwen2.5-1.5b-instruct-q4_k_m.gguf	1,04 ГБ

Модели вынесены из APK осознанно: 229 МБ ASR-графа и 1,04 ГБ LLM не проходят ни по лимиту магазина, ни по здравому смыслу обновлений. Качаются при первом запуске (LLM — автоматически), проверяются по размеру, а GGUF — ещё и по SHA-256. Если файла нет, приложение работает, но без LLM-пунктуации.

3. Декодирование аудио

На вход принимаются WAV, MP3, MP4/M4A/AAC и всё, что умеет системный MediaCodec. Результат всегда один: `FloatArray` в диапазоне −1…1, моно, 16 кГц.

WAV читается вручную

Парсер RIFF проходит чанки, читает `fmt ` и `data`, поддерживает PCM 8/16/24/32 бита, IEEE float 32/64 и `WAVE_FORMAT_EXTENSIBLE` (реальный формат берётся из SubFormat GUID). Каналы усредняются в моно. Такой путь не зависит от кодеков устройства и работает даже на «сырых» файлах из датасета.

MP3 / MP4 — через MediaCodec

MediaExtractor выбирает первую дорожку с MIME `audio/*` (видео игнорируется), MediaCodec декодирует её в PCM. Промежуточный WAV не создаётся. Поддержаны PCM16, PCM float и PCM8 на выходе декодера.

Ресемплинг

Линейная интерполяция к 16 кГц. Для речи этого достаточно, а стоимость нулевая. Ограничение честно записано в LIMITATIONS: на музыке и очень высоких частотах возможны алиасы; для ASR это не критично.

4. Нарезка на сегменты

Офлайн-модели из лидерборда плохо переносят длинный вход, а Whisper в sherpa-onnx вообще смотрит только первые 30 секунд. Поэтому аудио режется на куски не длиннее 25 секунд.

Как выбирается граница

- Сигнал делится на 20-мс рамки, для каждой считается RMS-энергия.
- Кандидаты на разрез — вторая половина куска (не меньше 50 % длины).
- Выбирается самая тихая рамка: обычно это пауза между словами.
- Полностью тихие куски пропускаются: пик меньше 1e-4.

Смещение куска сохраняется, поэтому все тайминги слов пересчитываются в абсолютное время файла ещё до склейки.

5. Распознавание речи: GigaAM v3

Модель: **GigaAM v3**, русский, MIT, ~229 МБ. Это RNNT-трансдьюсер NeMo, сконвертированный в ONNX. В APK-пайплайне используются `encoder.int8.onnx` (224,6 МБ), `decoder.onnx` и `joiner.onnx`.

Конфигурация распознавателя

Параметр	Значение
Тип модели	nemo_transducer
Признаки	fbank, 16 кГц, 80 измерений
Декодирование	greedy_search (без beam search)

Потоки	4 (numThreads)
Кусок	≤ 25 с

Greedy-декодер на каждом кадре берёт argmax по джойнеру: encoder даёт акустическое представление, decoder — состояние предсказателя, joiner смешивает их и выдаёт распределение по токенам. Blank-токены пропускаются, повторы схлопываются. Для русской речи это даёт WER 2,44 % при RTF 0,043 — быстрее реального времени в 23 раза.

Из токенов в слова

Модели отдают токены по-разному: GigaAM — по символу, Zipformer и Parakeet — SentencePiece с маркером ▁, Whisper — BPE. Модуль `WordTimestamps` приводит всё к словам:

- ▁ (U+2581) и Ġ считаются пробелом;
- служебные токены вида `<blk>`, `<sos/eos>` отбрасываются;
- пунктуация приклеивается к предыдущему слову, а не становится отдельным словом;
- если тайминги токенов пришли нулевые, длительность куска делится пропорционально длине токенов;
- если у токена нет длительности, конец оценивается по следующему токену или как 60 мс на символ.

На выходе — список `WordTimestamp(word, start, end)` в секундах от начала файла. Именно эти границы потом сравниваются с сегментами диаризации.

6. Диаризация и автоматическое число спикеров

На устройстве используется **sherpa-onnx OfflineSpeakerDiarization**: сегментатор pyannote 3.0 + экстрактор эмбеддингов ERes2Net base + быстрая кластеризация. Модели те же, что в оффлайн-бенчмарке, поэтому цифры из лидерборда переносятся на приложение с оговорками (разный режим auto-K).

6.1. Сегментация речи

pyannote segmentation 3.0 — это powerset-модель: на каждом кадре она предсказывает один из 7 классов, кодирующих до трёх одновременных говорящих (включая перекрытия). Окно — 10 секунд, сдвиг — 50 % (`windowShiftRatio = 0.5`). Затем срабатывает склейка: `minDurationOn = 0.3 c`, `minDurationOff = 0.5 c` — короткие «всплески» и паузы не превращаются в лишние сегменты.

6.2. Эмбеддинги

Каждое окно речи превращается в компактный L2-нормированный вектор (ERes2Net base, 37,8 МБ). Нормализация важна: дальше близость считается косинусом, и длина вектора не должна влиять на решение.

6.3. Кластеризация

sherpa-onnx использует быструю кластеризацию с порогом по косинусной близости. Ключевая деталь: в Kotlin-API нет публичного экстрактора эмбеддингов, поэтому нельзя посчитать gap-статистику, как в Python-пайплайне. Вместо этого приложение делает серию прогонов.

6.4. Auto-K: перебор порогов и «плато»

Порог — единственная ручка, которая определяет K. Низкий порог дробит спикеров (K завышается), высокий — склеивает всех в одного. Настоящее K обычно образует плато: несколько соседних порогов дают одинаковый результат. Алгоритм `SpeakerCountSweep`:

- прогнать диаризацию при порогах 0.4, 0.5, 0.6, 0.7, 0.8;
- взять K, который встретился чаще всего;
- при равенстве частот — K, ближайший к медиане серии;
- внутри плато выбрать прогон ближе к середине диапазона порогов.

Порог	0.4	0.5	0.6	0.7	0.8
Полученное K	3	2	2	2	1

Пример (иллюстративный): чаще всего встречается K = 2, плато — пороги 0.5–0.7, выбирается средний прогон 0.6.

Почему так, а не «умнее»: метод детерминированный, не требует обучаемых порогов и объясним в защите. На русских диалогах он даёт 75 % точного K (RU dev, 8 файлов) и 5/5 на подготовленных диалогах из датасета. На длинном AMI плато не образуется — это ограничение метода, а не сборки, и оно зафиксировано в LIMITATIONS.

6.5. Что делает исследовательский пайплайн иначе

В Python-бенчмарке (и в оффлайн-отчёте) auto-K считается через **gap-статистику**: для K = 1…12 считается качество кластеризации, затем K сравнивается с эталонным распределением случайных данных. Кластеризация — complete-linkage AHC по косинусу, окна 2 с/1 с. Это дороже по времени, но не требует перебора порогов и даёт более устойчивое K на длинных записях. На телефоне выбран компромисс: sweep быстрее и укладывается в бюджет памяти.

7. Назначение спикера каждому слову

Для каждого слова считается временное перекрытие с каждым сегментом диаризации. Побеждает сегмент с максимальным перекрытием:

```
overlap = min(word.end, seg.end) − max(word.start, seg.start)
```

Если слово попало в паузу и перекрытия нет, берётся ближайший по времени сегмент. Правило детерминированное и не делает лишних проходов по аудио. В Python-пайплайне на стыках реплик дополнительно проверялись эмбеддинги слова, но на Android экстрактор недоступен — осознанный компромисс.

Точность такого назначения на RU test: 82,8 % при auto-K и 90,3 % при oracle-K (484 слова). Ошибки сосредоточены на коротких словах в местах смены говорящего.

8. Склейка реплик и JSON

Слова собираются в реплики по трём правилам. Новая реплика начинается, если:

- сменился спикер;
- пауза между словами больше 1,2 с;
- текущая реплика уже длиннее 15 с (длинный монолог режется для LLM и чтения).

На выходе — ровно тот JSON, который ждёт заказчик:

```
{"transcript":[{"speaker_id":1,"start_ms":160,"end_ms":3640,"text":"Ну что ж, будем дальше
исследовать мир китайской кухни."}]}
```

`speaker_id` — целое число от 1 до K. Нумерация устойчивая: первый появившийся спикер получает 1. Экспорт идёт через системный диалог сохранения файла, поэтому JSON можно положить в любое место на устройстве.

9. Локальная LLM и системный промпт

Пунктуацию и регистр правит **Qwen2.5 1.5B Instruct Q4_K_M** — квантованная GGUF-модель на 1,04 ГБ, запущенная через llama.cpp на CPU. Она не переписывает смысл: на вход даётся одна реплика, на выход — та же реплика с пунктуацией.

Параметры генерации

Параметр	Значение	Почему
Контекст	2048 токенов	хватает на реплику + промпт
Потоки	4	баланс скорости и нагрева
temperature	0.0	воспроизводимость
top_p / top_k	1.0 / 1	жадная генерация
seed	42	одинаковый результат
max_tokens	128…512	зависит от длины реплики

Промпт хранится в SharedPreferences и редактируется прямо на главном экране: карточка «Промпт LLM» → «Изменить». Это позволяет менять поведение (например, «сохраняй разговорную лексику» или «пиши цифры словами») без пересборки APK. Кнопка «Сбросить» возвращает заводскую инструкцию.

Заводской промпт

> *«Ты — редактор расшифровки русской речи. На вход даётся одна реплика. Расставь знаки препинания, заглавные буквы и исправь очевидные ошибки распознавания. Не добавляй новую информацию, не меняй смысл и не пересказывай. Верни ТОЛЬКО исправленный текст реплики, без кавычек, пояснений и markdown.»*

Страховка

LLM может вернуть пустоту, мусор или текст без пунктуации. Тогда включается `LocalTextPolish`: убирает кавычки по краям, схлопывает пробелы, поднимает первую букву, поднимает регистр после `.` `!` `?` … и добавляет точку в конце. Текст никогда не остаётся «потоком слов». Слова и тайминги при этом не меняются — правка касается только строки текста.

Стоимость

1.5B Q4_K_M занимает ~1,4–1,6 ГБ RSS и добавляет примерно 25–40 секунд на 56-секундный диалог из 4 реплик (замер на iQOO 15R). Именно поэтому полоса прогресса после диаризации идёт медленно: это не зависание, а генерация по каждой реплике.

Платформенное ограничение

Бесплатный AAR `llama-android` содержит только `arm64-v8a`. На x86_64-эмуляторе LLM не загружается, и приложение автоматически отключает автоскачивание, оставляя rule-based fallback. Для реального устройства нужен arm64.

10. Метрики и результаты

Что означают цифры

Метрика	Смысл
WER	доля ошибок в словах: (S+D+I)/N, меньше — лучше
CER	то же по символам
DER	(confusion + miss + false alarm) / длительность речи
K exact	доля файлов, где число спикеров угадано точно
K MAE	средняя абсолютная ошибка в числе спикеров
Word accuracy	доля слов с правильно назначенным спикером
RTF	время обработки / длительность аудио; < 1 — быстрее реального времени

ASR: FLEURS ru + Golos crowd, 200+200 фраз

Модель	WER avg	RTF	RAM
GigaAM v3	**2,44 %**	0,043	337 МБ
Zipformer-RU	3,55 %	0,019	401 МБ
Parakeet TDT 0.6B	7,39 %	0,064	988 МБ
Whisper large-v3-turbo	18,18 %	0,382	2288 МБ

Диаризация: 1 AMI + 1 Vox + 8 RU, DER усреднён по трём датасетам

Режим	Модель	DER	K exact
auto K	CAM++ zh/en	33,20 %	50 %
oracle K	ERes2Net en	27,28 %	—
sweep (Kotlin)	ERes2Net base	27,85 % (RU dev)	75 %

Полный пайплайн на русских диалогах (5 файлов, ~8,3 мин)

Режим	DER	Word accuracy	K exact
auto K	**18,97 %**	**88,11 %**	5/5
oracle K	18,97 %	88,11 %	5/5

DER 18,97 % на диалогах заметно лучше 33 %, потому что там чистый русский материал, два спикера и нормальные паузы. Разброс по файлам большой: 5,1–46,1 % — один диалог с перекрытиями и короткими репликами заметно сложнее.

Тесты

71 JVM-тест на ASR, 25 на диаризацию и 63 на Android-приложение — всего **159 тестов**, 0 падений. Отдельно проверены формат JSON (целочисленный speaker_id), fallback без LLM и целостность файлов моделей.

11. Проверка на реальном устройстве

iQOO 15R (Android 16, arm64-v8a), диалог `dialogs_ru_002.wav` из датасета: 56,5 с, два спикера. Модели заранее залиты в filesDir, чтобы проверять пайплайн, а не сеть.

Показатель	Результат
Длительность	56,5 с
Спикеров	2 (K угадан)
Слов	119
Реплик	4
Пунктуация	корректная, от Qwen2.5
Время распознавания	≈ 50–60 с вместе с LLM

Границы реплик: приложение против разметки

Спикер	Приложение	Разметка	Расхождение
1	0,16–3,64	0,00–4,23	0,16 / 0,59 с
2	4,76–18,14	4,68–19,16	0,08 / 1,02 с
1	19,84–35,26	19,61–35,32	0,23 / 0,06 с
2	36,02–55,80	35,77–56,03	0,25 / 0,23 с

Расхождение — по началу и концу сегмента; смена говорящего определена верно во всех случаях.

12. Ограничения

- LLM работает только на arm64: бесплатный AAR llama.cpp не содержит x86_64.
- Модель LLM весит 1,04 ГБ и требует интернета при первом запуске.
- Auto-K через sweep нестабилен на длинных записях: на AMI плато не образуется.
- Перекрытия речи обрабатываются частично: слово получает одного спикера.
- Ресемплинг линейный — для музыки и широкополосного шума не оптимален.
- Русский — приоритет: на английском WER у GigaAM заметно выше.
- Нет экспорта RTTM и пакетной обработки очереди файлов.

Все ограничения продублированы в docs/LIMITATIONS.md, методика — в docs/PROTOCOL.md, цифры — в docs/TEST_RESULTS.md.

13. Где что лежит в коде

Шаг	Файл
Декодирование	com/nuclearhack/transcription/AudioDecoder.kt
Нарезка	com/nuclearhack/transcription/AudioChunker.kt
ASR	com/nuclearhack/transcription/GigaAmTranscriber.kt
Токены → слова	com/nuclearhack/transcription/WordTimestamps.kt
Диаризация	com/nuclearhack/diarization/SherpaDiarizer.kt

Auto-K	com/nuclearhack/diarization/SpeakerCountSweep.kt
Спикер словам	com/nuclearhack/diarization/WordSpeakerAssigner.kt
Реплики	com/nuclearhack/diarization/UtteranceBuilder.kt
LLM	com/nuclearhack/transcription/LlamaCppLocalLlm.kt
Промпт	com/example/baseapp/transcriber/LlmPromptStore.kt
Оркестратор	com/example/baseapp/transcriber/GigaAmTranscriber.kt

Отчёт описывает сборку из коммита main на 8 ноября 2026: APK 93 863 438 байт, SHA-256 8107B440…A406.